JC4001: Distributed Systems

Topic 1: Introduction

Xiaonan Liu
xiaonan.liu@abdn.ac.uk

Course Arrangements and Assessment

- Lectures (theory classes) 4 times a week
- Practicals (lab classes) 4 times a week
- In Class Quiz 20%
- Assignment 30%

  Have been released, submission in MyAberdeen

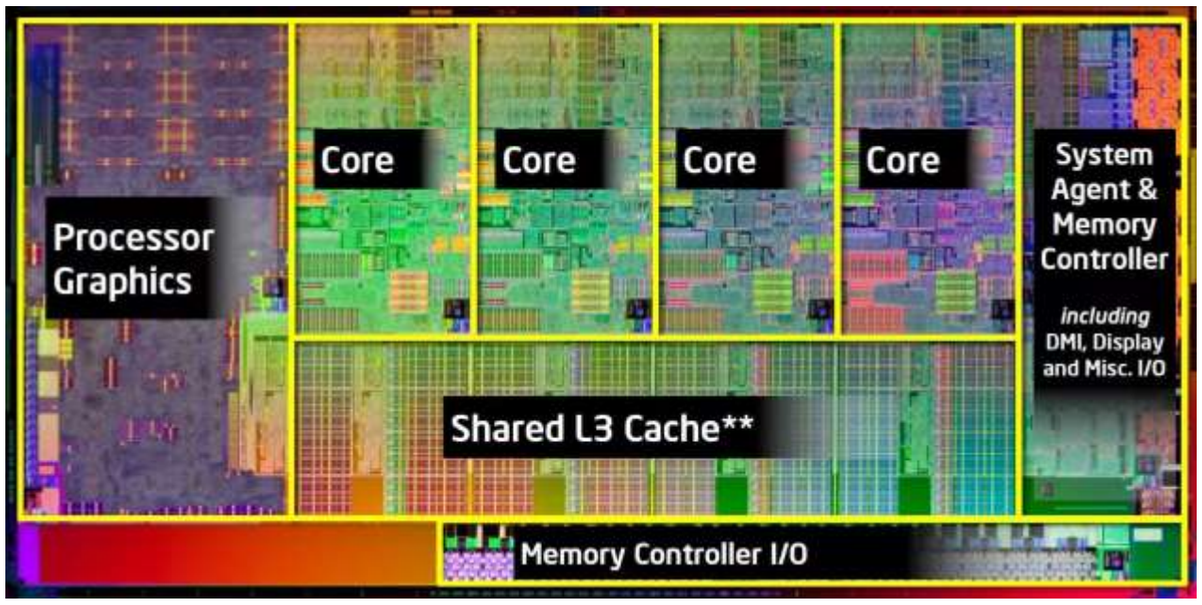
  Feedback no longer than four weeks after the deadline
- Exam 50%

Reading

- Distributed systems by Maarten van Steen Andrew S. Tanenbaum.
- Distributed and cloud computing: from parallel processing to the internet of things. by Hwang, Kai, Jack Dongarra, and Geoffrey C. Fox.

Evolution of Computer Systems

- 1945 - 1985: Large, expensive computers operating independently.
- Mid-1980s Onwards: Major advancements began to reshape computing.
  - Powerful Microprocessors
    - Transition from 8-bit to 16-, 32-, and 64-bit CPUs.
    - Multicore CPUs offer immense computing power at a fraction of the cost.
    - Machines have the computing power of mainframes from 30 or 40 years ago, but they cost a tiny fraction—about1/1000th—of what those mainframes did.



Evolution of Computer Systems

- 1945 - 1985: Large, expensive computers operating independently.
- Mid-1980s Onwards: Major advancements began to reshape computing.
  - High-Speed Computer Networks
    - **LANs (Local-Area Networks)**: thousands of machines within a building to connect and transfer small amounts of information in just a few microseconds. Larger amounts of data can be moved between machines at rates of billions of bits per second.

    - **WANs (Wide-Area Networks)**: connect hundreds of millions of machines around the world, with speeds ranging from tens of thousands to hundreds of millions of bits per second, and even more

Evolution of Computer Systems

- Miniaturization and Accessibility
  - Smartphones: Powerful, sensor-rich, networked devices. Packed with sensors, lots of memory, and a powerful multicore CPU, these devices are nothing less than full-fledged computers. Of course, they also have networking capabilities.

  - Nano Computers: Small, affordable single-board computers. These small, single-board computers, often the size of a credit card, can easily offer near-desktop performance. Well-known example is Raspberry Pi.

Evolution of Computer Systems

- Increasing Digitalization
  - As digitalization of our society continues, we become increasingly aware of how many computers are actually being used, regularly embedded into other systems such as cars, airplanes, buildings, bridges, the power grid, and so on.
  - Monitored and controlled by networked computer systems: Unfortunately, this awareness often increases when these systems are hacked. For instance, in 2021, a ransomware attack effectively shut down a fuel pipeline in the United States. In this case, the computer system consisted of a mix of sensors, actuators, controllers, embedded computers, servers, etc., all brought together into a single system. What many of us do not realize, is that vital infrastructures, such as fuel pipelines, are monitored and controlled by networked computer systems.

Mobile networked computer

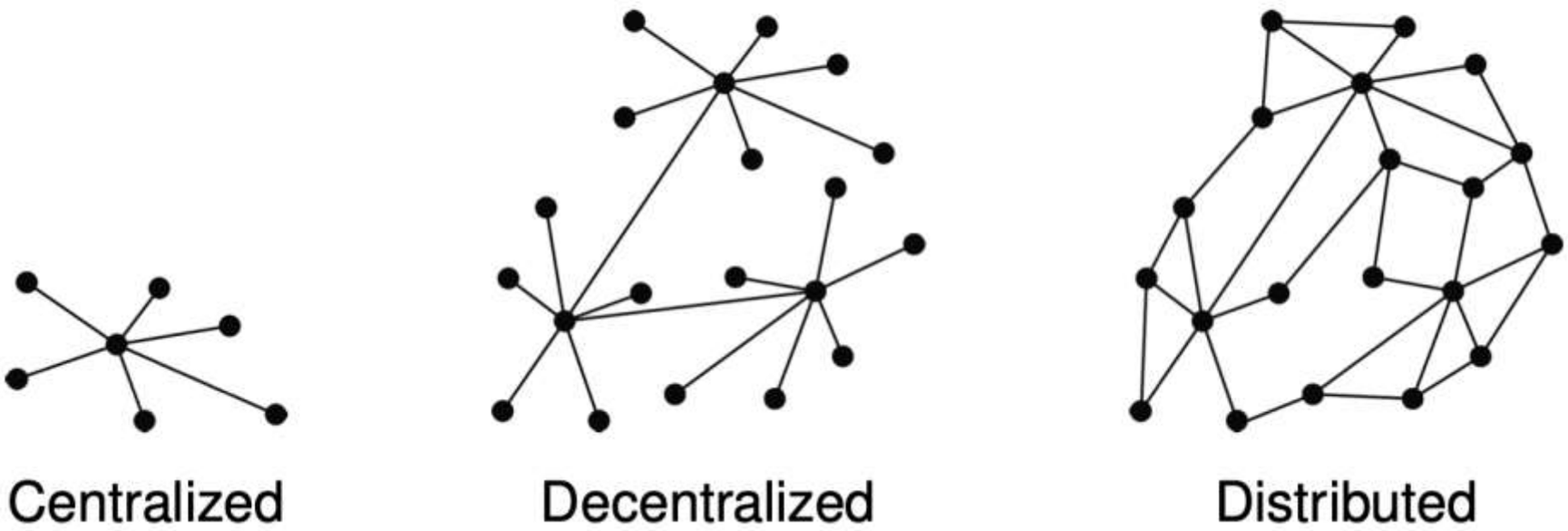
Distributed Systems

- Size
  - The size of a networked computer system may vary from a handful of devices, to millions of computers.

- Interconnection network
  - The interconnection network may be wired, wireless, or a combination of both. Moreover, these systems are often highly dynamic, in the sense that computers can join and leave, with the topology and performance of the underlying network almost continuously changing.
  - It is difficult to think of computer systems that are not networked. And as a matter of fact, most networked computer systems can be accessed from any place in the world because they are connected to the Internet. Studying and understanding these systems can quickly become very complex.

Decentralized vs. Distributed

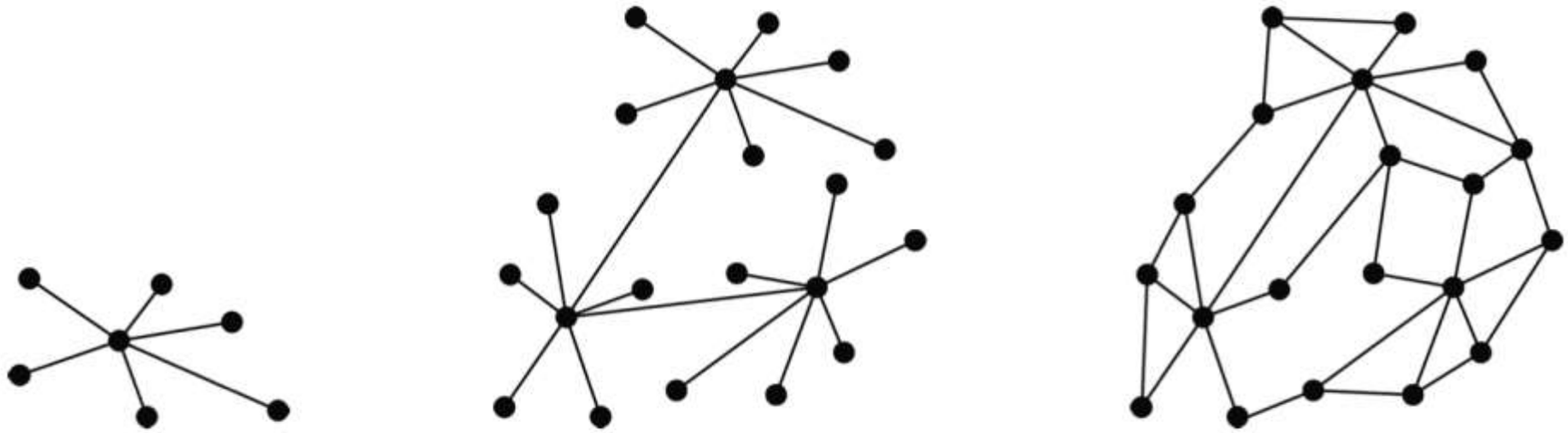
- The distinction is illustrated by three different organizations of networked computer systems, as shown in here, where each node represents a computer system and an edge a communication link between two nodes.



- Centralized System: A single central node connected to several peripheral nodes. All communication and control pass through a central point. If the central node fails, the entire system is disrupted.

Decentralized vs. Distributed

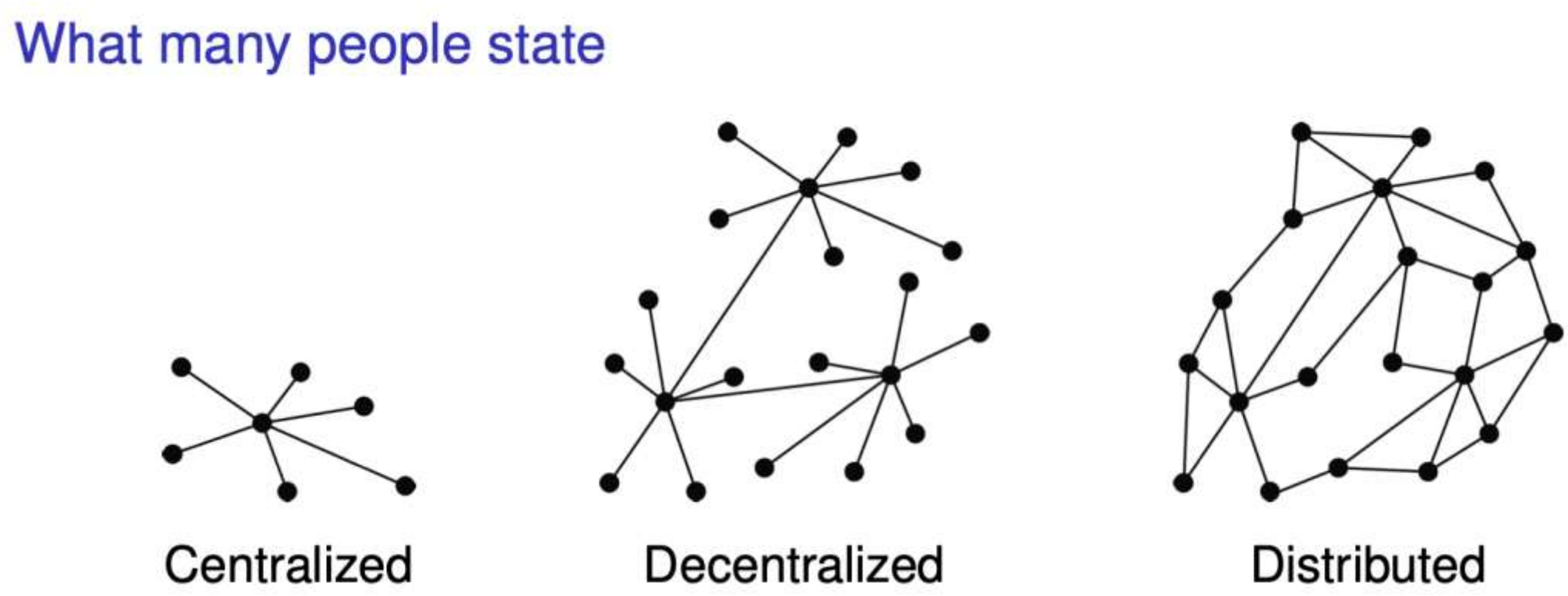
What many people state



Centralized Decentralized Distributed

- Decentralized System: Several central nodes each connected to their own set of peripheral nodes, with some connections between the central nodes. There are multiple central points, each responsible for a subset of the network. Failure of one central node does not necessarily disrupt the entire system. More resilient than a centralized system but still has critical points of failure.

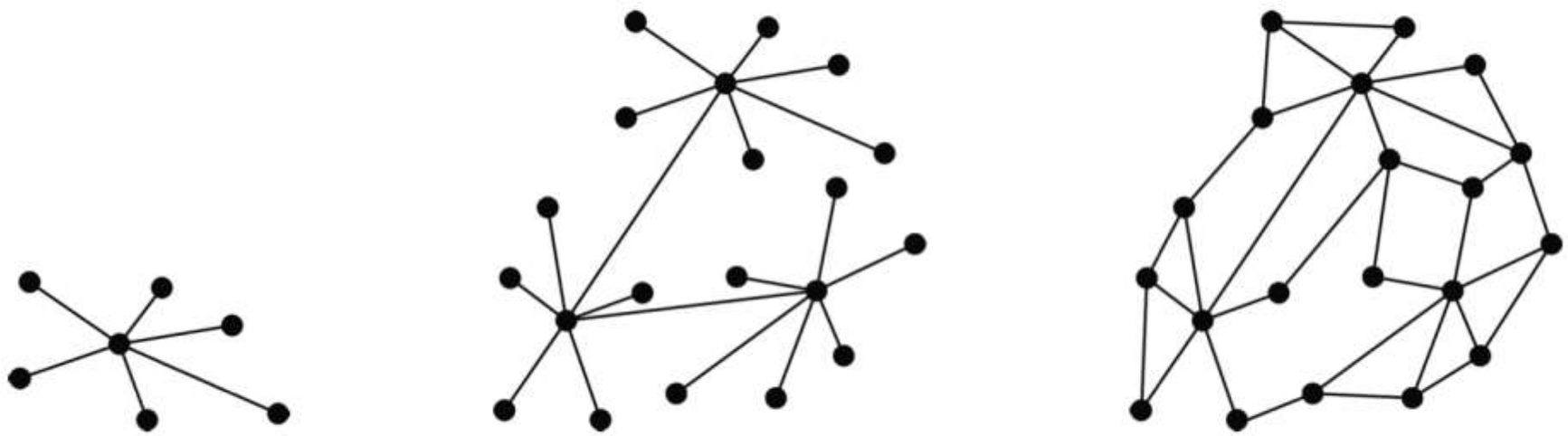
Decentralized vs. Distributed



- Distributed System: Multiple nodes interconnected in a web-like structure without a clear central point. Each node is connected to multiple other nodes, facilitating multiple paths for data to travel. Highly resilient as there is no single point of failure. Ideal for large-scale networks where reliability and redundancy are crucial.

Centralized, Decentralized, Distributed

What many people state



Centralized Decentralized Distributed

When does a decentralized system become distributed?

- Adding 1 link between two nodes in a decentralized system?
- Adding 2 links between two other nodes?
- In general: adding $k > 0$ links....?

The transition involves increasing connectivity and reducing dependency on central nodes, enhancing the network's resilience and fault tolerance.

Alternative approach

If we think of a networked computer system as a collection of computers connected in a network, we can ask ourselves how these computers even became connected to each other

- Two views on realizing distributed systems
  - Integrative view: there was a need to connect existing (networked) computer systems to each other. Typically, this happens when services running on a system need to be made available to users and applications that were not thought of before. This may happen, for example, when integrating financial services with project management services, as is often the case within a single organization.
  - Expansive view: that an existing system required an extension through additional computers. This view is the one most often related to the field of distributed systems. It entails expanding a system with computers to hold resources close to where those resources are needed. An expansion may also be driven by the need to improve dependability: if one computer fails, then there are others who can take over. An important type of expansion is when a service needs to be made available for remote users and applications, for example, by offering a Web interface or a smartphone application. This last example also shows that the distinction between an integrative and an expansive view is not a clear-cut.

Alternative approach

- Two definitions
  - A decentralized system is a networked computer system in which processes and resources are necessarily spread across multiple computers. They are mainly related to the integrative view of networked computer systems. They come to being because we want to connect systems, yet may be hindered by administrative boundaries.
  - For example, many applications in the artificial-intelligence domain require massive amounts of data for building reliable predictive models. Normally, data is brought to the high-performance computers that literally train models before they can be used. But when data needs to stay within the perimeter of an organization (and there can be many reasons why this is necessary), we need to bring the training to the data.
    - Example: Federated Learning: is implemented by a decentralized system, where the need for spreading processes and resources is dictated by administrative policies.

Alternative approach

- Two definitions
  - A distributed system is a networked computer system in which processes and resources are sufficiently spread across multiple computers. They are mainly related to the expansive view of networked computer systems. A well-known example is making use of e-mail services, such as Google Mail. What often happens is that a user logs into the system through a Web interface to read and send mails. More often, however, is that users configure their personal computer (such as a laptop) to make use of a specific mail client. To that end, they need to configure a few settings, such as the incoming and outgoing server. In the case of Google Mail, these are imap.gmail.com and smtp.gmail.com, respectively. Logically, it seems as if these two servers will handle all your mail. However, with an estimate of close to 2 billion users as of 2022, it is unlikely that only two computers can handle all their e-mails (which was estimated to be more than 300 billion per year, that is, some 10,000 mails per second).

Alternative approach

- Two definitions
  - Distributed System : Behind the scenes, the entire Google Mail service has been implemented and spread across many computers, jointly forming a distributed system. That system has been set up to make sure that so many users can process their mails (i.e., ensures scalability), but also that the risk of losing mail because of failures, is minimal (i.e., the system ensures fault tolerance). To the user, however, the image of just two servers is kept up (i.e., the distribution itself is highly transparent to the user). The distributed system implementing an e-mail service, such as Google Mail, typically expands (or shrinks) as dictated by dependability requirements, in turn, dependent on the number of its users. You can log in your Gmail at any computers.

Alternative approach

- Two definitions
  - As a final, much smaller distributed system, consider a setup based on a Network-Attached Storage device, also called a NAS. For domestic use, a typical NAS consists of 2–4 slots for internal hard disks. The NAS operates as a file server: it is accessible through a (generally wireless) network for any authorized device, and as such can offer services like shared storage, automated backups, streaming media, and so on. The NAS itself can best be seen as a single computer optimized for storing files, and offering the ability to easily share those files. The latter is important, and together with multiple users, we essentially have a setup of a distributed system. The users will be working with a set of files that are locally (i.e., from their laptop) easily accessible (in fact, seemingly integrated into the local file system), while also directly accessible by and for other users. Again, where and how the shared files are stored is hidden (i.e., the distribution is transparent). Assuming that sharing files is the goal, then we see that indeed a NAS can provide sufficient spreading of processes and resources.

Some common misconceptions

Centralized solutions do not scale: There appears to be a misconception that centralized solutions cannot scale and are always associated with introducing a single point of failure. Both reasons are often enough to dismiss centralized solutions as a viable option when designing distributed systems.

Make distinction between logically and physically centralized.

- logically centralized: A logically centralized solution can actually be implemented in a highly scalable and distributed manner.
- physically (massively) distributed
- decentralized across several organizations

•Example: The root of the Domain Name System: DNS is logically centralized because there is a single root for the name hierarchy, but physically, it is distributed across many servers around the world. The root node of DNS is not just a single server. In reality, it's implemented by 13 different root servers, each of which is a large cluster of computers. This redundancy means that the root node is not a single point of failure. DNS's physical organization ensures high availability and resilience. It would take an extraordinary effort to bring down all 13 root servers simultaneously, and so far, no such attempt has succeeded.

Some common misconceptions

Centralized solutions have a single point of failure

Generally not true (e.g., the root of DNS). A single point of failure is often:

- easier to manage
- easier to make more robust
- Centralized solutions are not inherently bad simply because they appear centralized. In fact, logically centralized solutions can sometimes be more manageable and efficient. The key point here is that while there may be a single point of failure in a logically centralized system, this point can be fortified against failures and attacks.
- centralized solutions have shown themselves to be extremely scalable and robust, often referred to as cloud-based solutions. These systems utilize sophisticated distributed techniques to ensure high performance and reliability. Even then, these solutions sometimes rely on a small set of physical machines to ensure performance and guarantee availability.

Important

There are many, poorly founded, misconceptions regarding scalability, fault tolerance, security, etc. We need to develop skills by which distributed systems can be readily understood so as to judge such misconceptions.

So sometimes, there is no strict boundary between centralized, decentralized, or distributed system. They can transfer from each other.

Perspectives on distributed systems

- Distributed systems are complex: Understanding distributed systems can be quite challenging, so it's important to look at them in a structured way. One of our major concerns is that there are so many explicit and implicit dependencies in distributed systems. Our approach is to understand distributed systems from a limited number, yet different perspectives
  - Architecture: It is crucial to start by understanding the overall structure of distributed systems. Discussing common organizations and styles helps to grasp how different parts of a system interact and depend on each other. This perspective provides a foundational understanding of system design.
  - Process: Processes are the core components of distributed systems. Understanding different types of processes, such as threads, clients, and servers, is essential because they form the software backbone of these systems. This perspective is vital for comprehending how distributed systems operate on a basic level.

Perspectives on distributed systems

- Distributed systems are complex:
  - Communication: In distributed systems, communication between processes is critical. Exploring how data is exchanged between processes helps in understanding how distributed systems maintain functionality and performance. This perspective is key to ensuring effective and efficient data transfer in a distributed environment.
  - Coordination: Processes in distributed systems must coordinate to function correctly. Coordination handles tasks such as managing access to shared resources and compensating for the lack of a global clock. This perspective is important for understanding the mechanisms that ensure distributed processes work together seamlessly.
  - Naming: Effective naming schemes are necessary to access processes and resources. Naming is crucial for locating and utilizing resources in a distributed system. This perspective focuses on how names are resolved to access points, which is fundamental for system functionality.

Perspectives on distributed systems

- Distributed systems are complex:
  - Consistency and replication: Distributed systems must be efficient and dependable. Replication of resources helps achieve this but introduces challenges like keeping all copies updated. This perspective examines the trade-offs between consistency, replication, and performance, which are essential for maintaining system integrity.
  - Fault tolerance: Distributed systems can experience partial failures. Understanding how systems mask failures and recover from them is critical for system reliability. This perspective is one of the most challenging but necessary areas to ensure distributed systems can handle and recover from failures effectively.
  - Security: The security perspective focuses on how to ensure authorized access to resources, but we may not have time to discuss this as a topic in the later lecture.

Overall Design Goals

- Support sharing of resources: the system should make it simple for users to access resources.
- Distribution transparency: It should hide the complexity of resources being spread across different network locations
- Openness: The system should be open, allowing for flexibility and easy integration with other systems
- Scalability: It should be able to grow and handle increasing amounts of work without performance issues

Sharing Resources

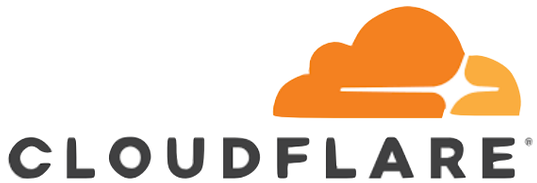
Cost Efficiency: It's cheaper to have one big storage system that everyone can use instead of each person buying and maintaining their own.

Collaboration: Connecting users and resources makes it easier for people to work together and share information. This is why the Internet, with its simple file-sharing protocols, has been so successful.

- Examples
- Groupware: Software that helps people work together, like for video calls or document sharing. This became especially important during the COVID-19 pandemic when many people started working from home.

- Peer-to-Peer Networks: such as BitTorrent, are excellent examples of resource sharing. Instead of downloading a file from a single server, P2P technology allows users to share parts of the file with each other. This method is particularly efficient for distributing large files, like movies or software updates, because it reduces the load on any single server and speeds up the download process by leveraging multiple sources.

Sharing Resources

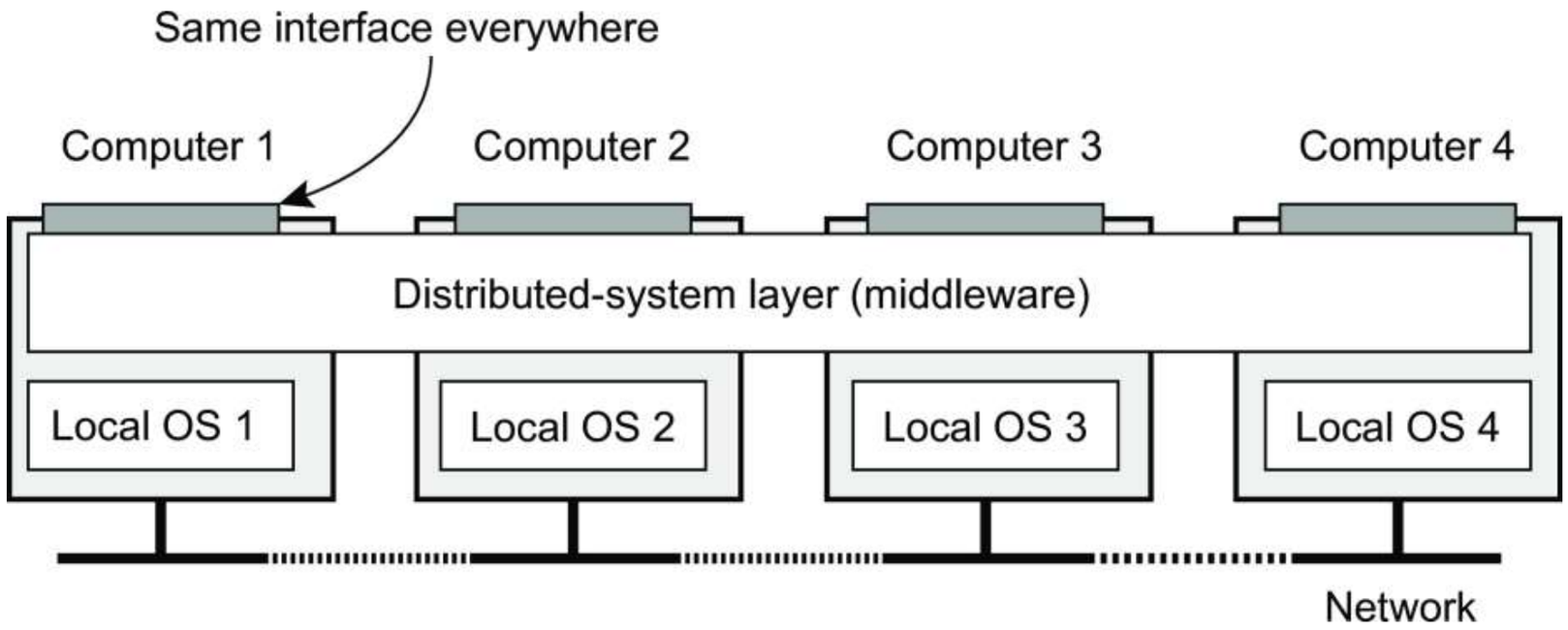
- Examples  
  - Consider email systems like Gmail and Outlook. These services manage your email storage, provide spam filtering, and ensure security, making email communication much simpler for users. Instead of each organization having to maintain their own email servers, these outsourced solutions handle everything, providing a seamless and reliable email experience.
  - Content Distribution Networks (CDNs) like Cloudflare or Akamai. These networks distribute website content across multiple servers located around the world. This means when you access a website, the content is delivered from the nearest server to you, improving load times and reliability. It's a smart way to handle high traffic and ensure that websites run smoothly regardless of user location.
- Seamless Integration: In modern distributed systems, sharing resources has become very common. For example, users can put files into a shared folder that everyone in the group can access, no matter where they are. This folder might be managed by a third party on the Internet.
- Special software makes this shared folder look just like any other folder on a user's computer, making it easy to use without worrying about where the data is actually stored.

Sharing Resources

Observation

“The network is the computer” It’s not just about individual machines anymore. It's about how these machines are linked together, sharing resources and working as a cohesive system. (quote from John Gage, then at Sun Microsystems)

Distribution Transparency



What is transparency?

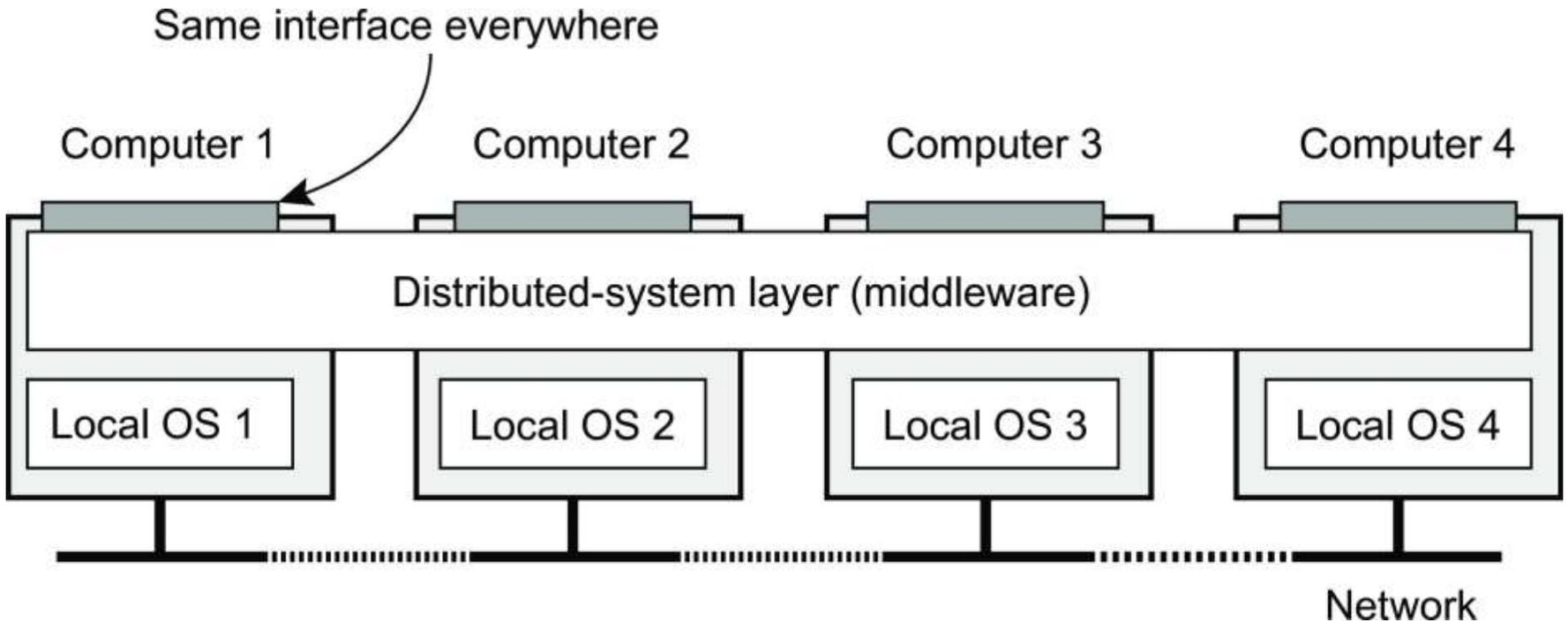
The phenomenon by which a distributed system attempts to hide the fact that its processes and resources are physically distributed across multiple computers, possibly separated by large distances.

It is the idea that a distributed system should hide the fact that its processes and resources are spread across multiple computers, which could be far apart. Essentially, it tries to make the system appear as a single, cohesive unit to users and applications, even though it's actually distributed.

It should hide the complexities of resources being distributed over a network.

For end users and applications, interacting with the system should feel seamless and straightforward

Distribution Transparency



How is Distribution Transparency Achieved?

Middleware: A crucial component that helps in achieving transparency

Middleware acts as a layer between applications and the underlying operating systems of the distributed computers.

It ensures that users see the same interface everywhere, regardless of the actual distribution of processes and resources.

Visual Example: Imagine we have four computers: Computer 1, Computer 2, Computer 3, and Computer 4. Each computer runs its own local operating system: Local OS 1, Local OS 2, Local OS 3, and Local OS 4.Above these operating systems, we have the distributed-system layer or middleware. This middleware ensures that all the computers present the same interface to applications and users, making the distribution of resources invisible to them.

Distribution Transparency

- Types

Transparency	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

Distribution Transparency

- Access transparency: Imagine a distributed system with different computer systems running different operating systems, each with its own file-naming conventions and file operations. Access transparency ensures that users and applications interact with these objects without needing to worry about these underlying differences. Essentially, it provides a uniform way to access resources, regardless of where they are located or how they are represented.

- Location transparency: It ensures that users cannot tell where an object is physically located within the system. This is achieved through logical names that do not reveal the physical location of resources. For instance, a URL (Uniform Resource Locator) like https://www.distributed-systems.net/ does not indicate where the server hosting the website is physically located. This type of transparency is crucial for hiding the physical distribution of resources from users.

Distribution Transparency

- Relocation transparency: It goes a step further by hiding the fact that resources may move within the system. For example, in cloud computing, services and data might be moved from one data center to another for load balancing or maintenance. With relocation transparency, users are unaware of these movements, ensuring uninterrupted access to resources.

- Migration transparency: It supports the mobility of processes and resources initiated by users. A common example is communication between mobile phones: users can continue their conversation seamlessly as they move from one place to another, without needing to know about the underlying handoffs between different network towers.

- Replication transparency: It deals with hiding the fact that multiple copies of a resource may exist in the system to increase availability and performance. For example, a distributed database might replicate data across multiple servers. Replication transparency ensures that users see a single logical entity, even though there are multiple physical copies. This also means that all replicas should have the same name and behave identically.

Distribution Transparency

- Concurrency transparency: It allows multiple users to share resources without interference. For example, two users might be accessing the same file or database simultaneously. Concurrency transparency ensures that the users do not need to worry about the other user's actions, maintaining consistency and integrity of the shared resource. This often involves mechanisms like locking and transactions to manage concurrent access.
- Failure transparency: It ensures that users and applications do not notice when a part of the system fails and that the system can recover automatically. This is one of the most challenging types of transparency to achieve. For example, if a web server goes down, a user should not experience a disruption in service because the system automatically redirects the request to a functioning server. The difficulty lies in distinguishing between temporary and permanent failures and ensuring seamless recovery.

Degree of Transparency

Aiming at full distribution transparency may be too much

- There are communication latencies that cannot be hidden
- Completely hiding failures of networks and nodes is (theoretically and practically) impossible
  - You cannot distinguish a slow computer from a failing one
  - You can never be sure that a server actually performed an operation before a crash
- Full transparency will cost performance, exposing distribution of the system
  - Keeping replicas exactly up-to-date with the master takes time
  - Immediately flushing write operations to disk for fault tolerance

Degree of Transparency

- Exposing distribution may be good
  - Making use of location-based services (finding your nearby friends): When dealing with location-based services, exposing distribution can be very useful. For instance, applications that help you find your nearby friends need to know and share your location. Here, transparency about the distribution of resources enhances functionality by providing relevant and localized information.
  - When dealing with users in different time zones: When users are in different time zones, being transparent about the distribution can improve user experience. For example, if you expect your electronic newspaper to be delivered at 7 AM local time while you are traveling, it helps to know that the service might not function as expected due to time zone differences.
  - Understanding System Behaviour: Exposing distribution can also make it easier for users to understand what's happening in the system. For example, if a server does not respond for a long time, it's better to inform the user that the server might be failing rather than leaving them in the dark. This transparency helps users make informed decisions, such as retrying the request or contacting support.

Degree of Transparency

Conclusion

While distribution transparency is a desirable goal, achieving full transparency is challenging and sometimes impractical. In many cases, it might be more effective to aim for partial transparency or explicitly expose certain aspects of the distribution to enhance user experience and system performance.

The key point is to balance the level of transparency with the needs of the system and its users. By doing so, we can design distributed systems that are both efficient and user-friendly.

Openness

Definition

A system that offers components that can easily be used by, or integrated into other systems. An open distributed system itself will often consist of components that originate from elsewhere.

To be open means that components should adhere to standard rules that describe the which service they provide. A general approach is to define services through interfaces using an Interface Definition Language (IDL). Interface definitions written in an IDL specify precisely the names of the functions that are available

What are we talking about?

Be able to interact with services from other open systems, irrespective of the underlying environment:

- Systems should conform to well-defined interfaces
- Systems should easily interoperate
- Systems should support portability of applications
- Systems should be easily extensible

Openness

- Interoperability characterizes the extent by which two implementations of systems or components from different manufacturers can co-exist and work together by merely relying on each other's services as specified by a common standard.

- Portability characterizes to what extent an application developed for a distributed system A can be executed, without modification, on a different distributed system B that implements the same interfaces as A.

- Extensible: For example, in an extensible system, it should be relatively easy to add parts that run on a different operating system, or even to replace an entire file system. Relatively simple examples of extensibility are plug-ins for Web browsers, but also those for Websites, such as the ones used for WordPress.

Openness

- Implementing openness: policies: rules, guidelines, and objectives that govern the behavior of a system. They define what should be done in order to achieve desired outcomes.
  - What level of consistency do we require for client-cached data?
  - Which operations do we allow downloaded code to perform?
  - Which QoS requirements do we adjust in the face of varying bandwidth?
  - What level of secrecy do we require for communication?
- Implementing openness: mechanisms: the methods, processes, and tools used to implement and enforce policies.
  - Allow (dynamic) setting of caching policies
  - Support different levels of trust for mobile code
  - Provide adjustable QoS parameters per data stream
  - Offer different encryption algorithms

  Example: In the case of Web caching, a browser should ideally provide facilities for only storing documents (i.e., a mechanism) and at the same time allow users to decide which documents are stored and for how long (i.e., a policy).

Openness

- Separation between policy and mechanism

Observation

In theory, strictly separating policies from mechanisms seems to be the way to go. However, there is an important trade-off to consider: the stricter the separation, the more we need to make sure that we offer the appropriate collection of mechanisms. In practice, this means that a rich set of features is offered, in turn leading to many configuration parameters. As an example, the popular Firefox browser comes with a few hundred configuration parameters. Just imagine how the configuration space explodes when considering large distributed systems consisting of many components. In other words, strict separation of policies and mechanisms may lead to highly complex configuration problems.

Openness

- Separation between policy and mechanism

Finding a balance

One option to alleviate these problems is to provide reasonable defaults, and this is what often happens in practice. An alternative approach is one in which the system observes its own usage and dynamically changes parameter settings. This leads to what are known as self-configurable systems. Nevertheless, the fact alone that many mechanisms need to be offered to support a wide range of policies often makes coding distributed systems very complicated. Hard-coding policies into a distributed system may reduce complexity considerably, but at the price of less flexibility. Hard coding requires the program's source code to be changed any time the input data or desired format changes.

Dependability

Basics

Dependability refers to the degree that a computer system can be relied upon to operate as expected. In contrast to single-computer systems, dependability in distributed systems can be rather complex due to partial failures: somewhere there is a component failing while the system as a whole still seems to meet the expectations. Although single-computer systems can also suffer from failures that do not appear immediately, having a potentially large collection of networked computer systems can be very complicated. In fact, one should assume that at any time, there are always partial failures occurring. An important goal of distributed systems is to mask those failures, as well as mask the recovery from those failures. This masking is the essence of being able to tolerate faults.

A component provides services to clients. To provide services, the component may require the services from other components ⇒ a component may depend on some other component.

Specifically

A component *C* depends on *C**if the correctness of *C*'s behavior depends on the correctness of *C**'s behavior. (Components are processes or channels, help in designing more reliable and dependable systems.)

Dependability

- Requirements related to dependability
  - Availability: It is defined as the property that a system is ready to be used immediately. In general, it refers to the probability that the system is operating correctly at any given moment and is available to perform its functions on behalf of its users. In other words, a highly available system is one that will most likely be working at a given instant in time.
  - Reliability: refers to the property that a system can run continuously without failure. In contrast to availability, reliability is defined in terms of a time interval instead of an instant in time. A highly reliable system is one that will most likely continue to work without interruption during a relatively long period of time. This is a minor but important difference when compared to availability. If a system goes down randomly on average for one millisecond every hour, it has an availability of more than 99.9999 percent, but is still unreliable. Similarly, a system that never crashes but is shut down for two specific weeks every August has high reliability but only 96 percent availability. The two are not the same.

Reliability $R(t)$ of component C

Conditional probability that C has been functioning correctly during $[0, t)$ given C was functioning correctly at the time $T = 0$.

Dependability

- Requirements related to dependability
  - Safety: Refer to the situation that when a system temporarily fails to operate correctly, no catastrophic/disastrous event happens. For example, many process- control systems, such as those used for controlling nuclear power plants or sending people into space, are required to provide a high degree of safety. If such control systems temporarily fail for only a very brief moment, the effects could be disastrous. Many examples from the past show how hard it is to build safe systems.

  - Maintainability: How easily a failed system can be repaired. A highly maintainable system may also show a high degree of availability, especially if failures can be detected and repaired automatically

Dependability

- Fault-tolerance is related to the following metrics

  - Mean Time To Failure (*MTTF*): The average time until a component fails.
  - Mean Time To Repair (*MTTR*): The average time needed to repair a component.
  - Mean Time Between Failures (*MTBF*): Simply *MTTF* + *MTTR*.

Dependability

- Failure, Error, Fault

Term	Description	Example
Failure	A component is not living up to its specifications	Crashed program
Error	Part of a component that can lead to a failure	Programming bug
Fault	Cause of an error	Sloppy programmer

- Transient/intermittent/Permanent Fault

Transient faults occur once and then disappear. If the operation is repeated, the fault goes away. For example, a bird flying through the beam of a microwave transmitter may cause lost bits on some network. If the transmission times out and is retried, it will probably work the second time.

An intermittent fault occurs, then disappear on its own accord, then reappears, and so on. A loose contact on a connector will often cause an intermittent fault.

A permanent fault is one that continues to exist until the faulty component is replaced. Burnt-out chips, software bugs, and disk-head crashes are examples of permanent faults.

Dependability

- Handling Faults

Term	Description	Example
Fault prevention	Prevent the occurrence of a fault	Don’t hire sloppy programmers
Fault tolerance	Build a component and make it mask the occurrence of a fault	Build each component by two independent programmers
Fault removal	Reduce the presence, number, or seriousness of a fault	Get rid of sloppy programmers
Fault forecasting	Estimate current presence, future incidence, and consequences of faults	Estimate how a recruiter is doing when it comes to hiring sloppy programmers

Security

Observation

Dependable systems are also required to provide security, especially in terms of confidentiality and integrity. A distributed system that is not secure, is not dependable.

What we need

- Confidentiality: information is disclosed only to authorized parties
- Integrity: Ensure that alterations to assets of a system can be made only in an authorized way

Confidentiality and integrity are directly coupled to authorized disclosure and access of information and resources. In any computer system, authorization is done by checking whether an identified entity has proper access rights. In turn, this means that the system should know it is indeed dealing with the proper entity.

Authorization, Authentication, Trust

- Authentication: verifying the correctness of a claimed identity
- Authorization: does an identified entity has proper access rights?
- Trust: one entity can be assured that another will perform particular actions according to a specific expectation

Security

Keeping it simple

It’s all about encrypting and decrypting data using security keys.

The easiest way of considering a security key K is to see it as a function operating on some data.

Notation

K (*data*) denotes that we use key *K* to encrypt/decrypt *data*.

Security

- Symmetric Cryptosystem

  Encryption and decryption takes place with a single key.

  With encryption key $E_K(data)$ and decryption key $D_K(data)$:

  if $data = D_K(E_K(data))$ *then* $D_K = E_K$. Note: encryption and descryption key are the same and should be kept secret.

- Asymmetric Cryptosystem

  Distinguish a public key $PK(data)$ and a private (secret) key $SK(data)$.

  - Encrypt message from *Alice* to *Bob*: $data = \underbrace{SK_{bob}(\overbrace{PK_{bob}(data)}^{\text{Sent by Alice}})}_{\text{Action by Bob}}$
  - Sign message for *Bob* by *Alice*:
    $[data, \underbrace{data \stackrel{?}{=} PK_{alice}(SK_{alice}(data))}_{\text{Check by Bob}}] = \underbrace{[data, SK_{alice}(data)]}_{\text{Sent by Alice}}$

Security

- Secure Hashing

Practical placement of digital signatures is generally more efficient by a hash function.

In practice, we use secure hash functions: *H*(*data*) returns a fixed-length string.

- Any change from *data* to *data**will lead to a completely different string *H*(*data**).
- Given a hash value, it is computationally impossible to find a *data* with

$h = H(data)$

- Practical digital signatures

Sign message for *Bob* by *Alice*:

$$[data, \underbrace{H(data) \stackrel{?}{=} PK_{alice}(sgn)}_{\text{Check by Bob}}] = \underbrace{[data, H, sgn = SK_{alice}(H(data))]}_{\text{Sent by Alice}}$$

Security

- Cryptography in Distributed System
  - Secure Channel
    - Ensure secure communication between two parties.
    - Example: https for secure web access
  - Blockchain: Chain of blocks with hashed data
    - Each block B_i contains a hash of its data.
    - Hash of block B_i is included in block B_{i+1}
    - Modifying any block requires changes to all subsequent blocks
  - Delegating access right
    - Scenario: Alice delegates rights to Bob, who delegates to David.
    - Verification: System can check David's authorization without querying Alice each time.
  - Multiparty computation
    - Compute a value jointly without revealing individual data.
    - Example: Secure vote counting without knowing individual votes.

Scalability

Observation

Many developers of modern distributed systems easily use the adjective “scalable” without making clear why their system actually scales.

At least three components

- Number of users or processes (size scalability)
- Maximum distance between nodes (geographical scalability)
- Number of administrative domains (administrative scalability)

Observation

Most systems account only, to a certain extent, for size scalability. Often a solution: multiple powerful servers operating independently in parallel. Today, the challenge still lies in geographical and administrative scalability.

Scalability

- Size Scalability

If more users or resources need to be supported, we will meet the limitations of centralized services. For example, many services are centralized in the sense that they are implemented by a single server running on a specific machine in the distributed system. In a more modern setting, we may have a group of collaborating servers co-located on a cluster of tightly coupled machines physically placed at the same location. The problem with this scheme is obvious: the server, or group of servers, can simply become a bottleneck when it needs to process an increasing number of requests. To illustrate how this can happen, let us assume that a service is implemented on a single machine.

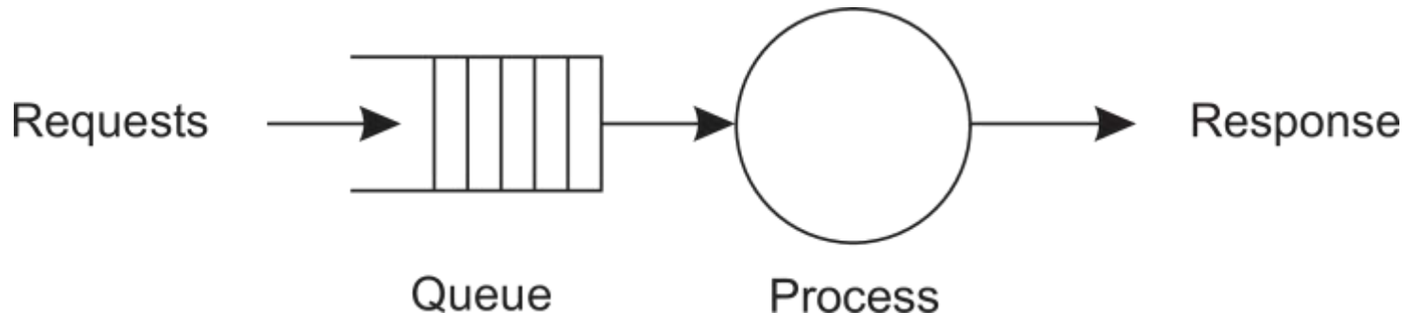
Root causes for scalability problems with centralized solutions

- The computational capacity, limited by the CPUs
- The storage capacity, including the transfer rate between CPUs and disks
- The network between the user and the centralized service

Scalability

- Formal Analysis

A centralized service can be modeled as a simple queuing system



Assumptions and notations

- The queue has infinite capacity ⇒ arrival rate of requests is not influenced by current queue length or what is being processed.
- Arrival rate requests: λ
- Processing capacity service: μ requests per second

Fraction of time having k requests in the system $\quad p_k = (1 - \frac{\lambda}{\mu})(\frac{\lambda}{\mu})^k$

Scalability

- Formal Analysis

  Utilization U of a service is the fraction of time that it is busy

$$U = \sum_{k>0} p_k = 1 - p_0 = \frac{\lambda}{\mu} \Rightarrow p_k = (1-U)U^k$$

  Average number of requests in the system

$$\overline{N} = \sum_{k\geq 0} k \cdot p_k = \sum_{k\geq 0} k \cdot (1-U)U^k = (1-U)\sum_{k\geq 0} k \cdot U^k = \frac{(1-U)U}{(1-U)^2} = \frac{U}{1-U}$$

  Average throughput

$$X = \underbrace{U \cdot \mu}_{\text{server at work}} + \underbrace{(1-U)\cdot 0}_{\text{server idle}} = \frac{\lambda}{\mu} \cdot \mu = \lambda$$

Scalability

Start with the original series:

$$S(U) = \sum_{k=0}^{\infty} U^k = \frac{1}{1-U}$$

Differentiate $S(U)$ with respect to U:

$$\frac{d}{dU} S(U) = \frac{d}{dU} \left(\frac{1}{1-U} \right)$$

The right-hand side becomes:

$$\frac{1}{(1-U)^2}$$

Now differentiate the left-hand side, remembering to apply the power rule for the terms inside the summation:

$$\frac{d}{dU} \left(\sum_{k=0}^{\infty} U^k \right) = \sum_{k=1}^{\infty} k \cdot U^{k-1} = \frac{1}{(1-U)^2}$$

Thus, we obtain:

$$\sum_{k=1}^{\infty} k \cdot U^{k-1} = \frac{1}{(1-U)^2}$$

3. Multiply Both Sides by U

Since our original summation has the form $\sum_{k \geq 0} k \cdot U^k$, multiply both sides of the equation by U to restore the U^k term on the left-hand side:

$$U \cdot \sum_{k=1}^{\infty} k \cdot U^{k-1} = U \cdot \frac{1}{(1-U)^2}$$

The left-hand side becomes the desired summation:

$$\sum_{k \geq 0} k \cdot U^k = \frac{U}{(1-U)^2}$$

Scalability

- Formal Analysis

Response time: total time take to process a request after submission

$$R = \frac{\overline{N}}{X} = \frac{S}{1-U} \Rightarrow \frac{R}{S} = \frac{1}{1-U}$$

with $S = \frac{1}{\mu}$ being the service time.

Observations

- If *U* is small, response-to-service time is close to 1: a request is immediately processed
- If *U* goes up to 1, the system comes to a grinding halt.
  Solution: decrease *S*.

Scalability

Problems with geographical scalability

- Cannot simply go from LAN to WAN: many distributed systems assume synchronous client-server interactions: client sends request and waits for an answer. Latency may easily prohibit this scheme.
- WAN links are often inherently unreliable: simply moving streaming video from LAN to WAN is bound to fail.
- Lack of multipoint communication, so that a simple search broadcast cannot be deployed. Solution is to develop separate naming and directory services (having their own scalability problems).

Scalability

Problems with administrative scalability

Essence

Conflicting policies concerning usage (and thus payment), management, and security

Examples

- Computational grids: share expensive resources between different domains.
- Shared equipment: how to control, manage, and use a shared radio telescope constructed as large-scale shared sensor network?

Exception: several peer-to-peer networks

- File-sharing systems (based, e.g., on BitTorrent)
- Peer-to-peer telephony (early versions of Skype)
- Peer-assisted audio streaming (Spotify)

Note: end users collaborate and not administrative entities.

Scalability: Techniques

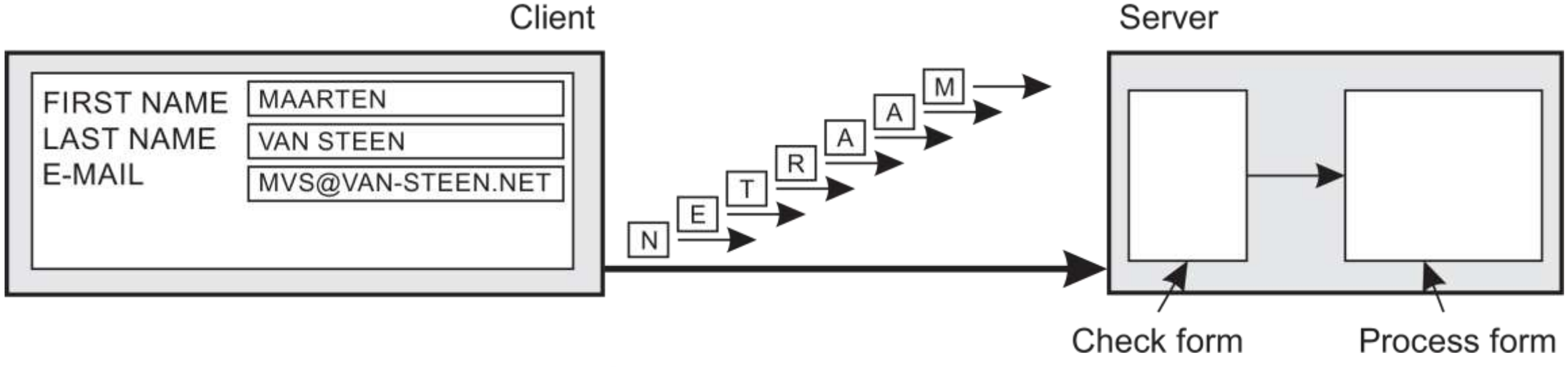
- In most cases, scalability problems in distributed systems appear as performance problems caused by limited capacity of servers and network. Simply improving their capacity is often a solution (e.g., by increasing memory, upgrading CPUs, or replacing network modules), referred to as scaling up. When it comes to scaling out, that is, expanding the distributed system by essentially deploying more machines, there are basically only three techniques we can apply: hiding communication latencies, distribution of work, and replication

Hide communication latencies: try to avoid waiting for responses to remote-service requests as much as possible

- Make use of asynchronous communication
- Have separate handler for incoming response
- Problem: not every application fits this model

Scalability: Techniques

Facilitate solution by moving computations to client



Traditional Server-Centric Processing

Client Side:

The client fills out a form with fields such as First Name, Last Name, and Email.

Communication:

The form data is sent from the client to the server.

Each piece of data (e.g., "MAARTEN", "VAN STEEN", "MVS@VAN-STEEN.NET") is sent to the server for validation and processing.

Server Side:

The server receives the data and performs the validation checks (e.g., ensuring the email is in a valid format).

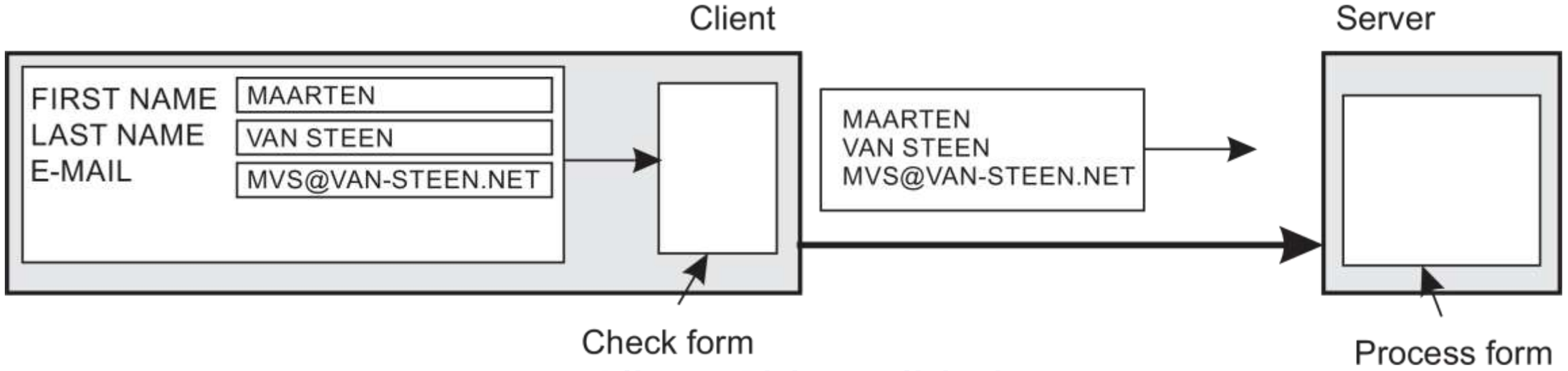
Once the form is checked and validated, the server processes the form data accordingly.

In this case, the server handles both checking and processing, which can create a bottleneck, especially if there are many clients sending requests simultaneously.

This approach can lead to increased server load and higher latency.

Scalability: Techniques

Facilitate solution by moving computations to client



Client-Side Validation

Client Side:

The client still fills out the form with the same fields (First Name, Last Name, Email). However, the client now performs the validation checks locally (e.g., ensuring the email is in a valid format). Once the form is validated by the client, it prepares the data for submission. In this case, only the validated data is sent to the server in one package, reducing the amount of data transferred and the number of interactions between the client and server.

Server Side:

The server receives the already validated form data. The server can directly process the form without needing to perform validation checks.

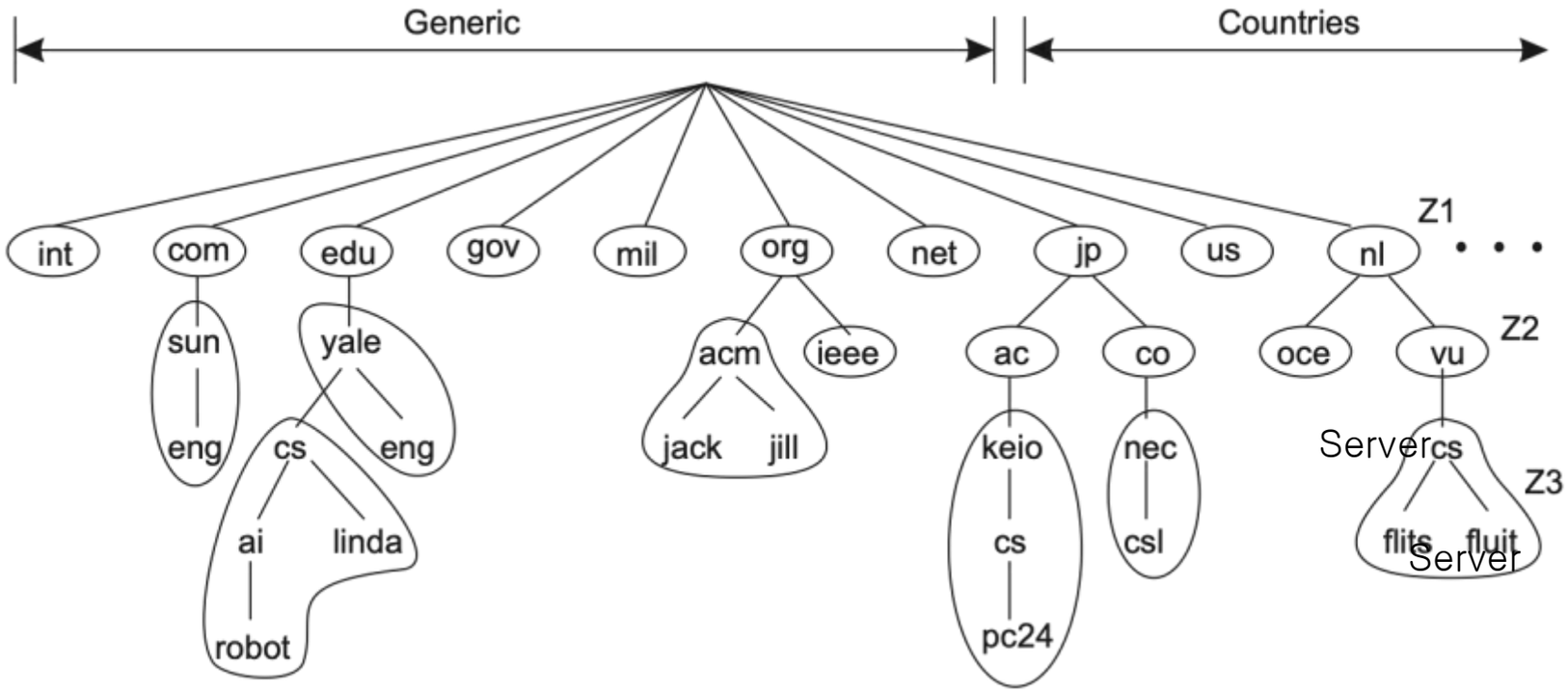
Benefits:

By moving the validation checks to the client, the server is relieved of these tasks, reducing its load. This can result in faster response times and more efficient overall system performance. The server can handle more requests simultaneously, improving scalability.

Scalability: Techniques

Partition data and computations across multiple machines

- Move computations to clients (Java applets and scripts)
- Decentralized naming services (DNS)



Example: flits.cs.vu.nl

The DNS name space is hierarchically organized into a tree of domains, which are divided into nonoverlapping zones. The names in each zone are handled by a single name server. One can think of each path name being the name of a host on the Internet, and is thus associated with a network address of that host. Basically, resolving a name means returning the network address of the associated host. Consider, for example, the name flits.cs.vu.nl. To resolve this name, it is first passed to the server of zone Z1 which returns the address of the server for zone Z2, to which the rest of the name, flits.cs.vu, can be handed. The server for Z2 will return the address of the server for zone Z3, which is capable of handling the last part of the name, and will return the address of the associated host. These examples illustrate how the naming service as provided by DNS, is distributed across several machines, thus avoiding that a single server has to deal with all requests for name resolution.

Scalability: Techniques

Partition data and computations across multiple machines

- Decentralized information systems (WWW)

To most users, the Web appears to be an enormous document-based information system, in which each document has its own unique name in the form of a URL. Conceptually, it may even appear as if there is only a single server. However, the Web is physically partitioned and distributed across a few hundreds of millions of servers, each handling often a number of Websites, or parts of Websites. The name of the server handling a document is encoded into that document's URL. It is only because of this distribution of documents that the Web has been capable of scaling to its current size. Yet, note that finding out how many servers provide Web-based services is virtually impossible: A Website today is so much more than a few static Web documents.

Scalability: Techniques

Replication and caching: Make copies of data available at different machines

- Replicated file servers and databases
- Mirrored Websites
- Web caches (in browsers and proxies)
- File caching (at server and client)

Scalability: Techniques

Applying replication is easy, except for one thing

- Having multiple copies (cached or replicated), leads to inconsistencies: modifying one copy makes that copy different from the rest.
- Always keeping copies consistent and in a general way requires global synchronization on each modification.
- Global synchronization precludes large-scale solutions.

Observation

If we can tolerate inconsistencies, we may reduce the need for global synchronization, but tolerating inconsistencies is application dependent.

Classification of Distributed Systems

What they are being developed?

What they are used for?

- High Performance Computing
  - Cluster computing: the underlying hardware consists of a collection of similar compute nodes, interconnected by a high-speed network, often alongside a more common local-area network for controlling the nodes. In addition, each node generally runs the same operating system.
  - Grid computing: This subgroup consists as decentralized systems that are often constructed as a federation of computer systems, where each system may fall under a different administrative domain, and may be very different when it comes to hardware, software, and deployed network technology.
- General Information processing
- Pervasive Computing (IoT)

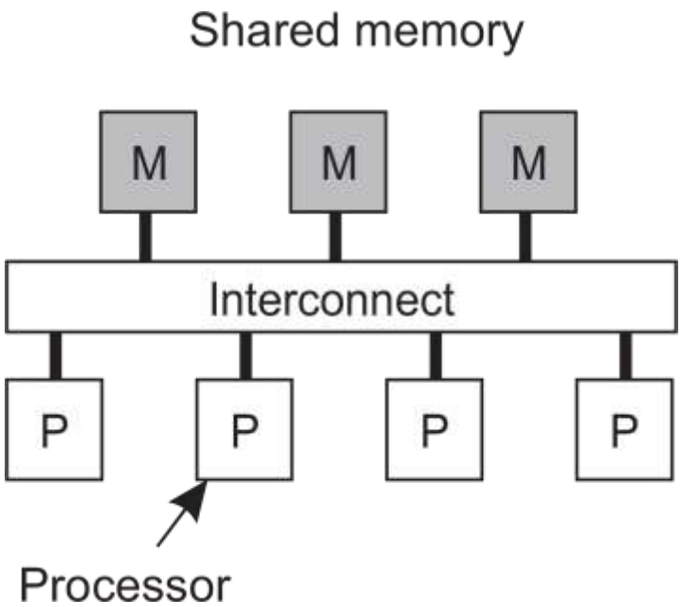
High-performance distributed computing

- Parallel Computing

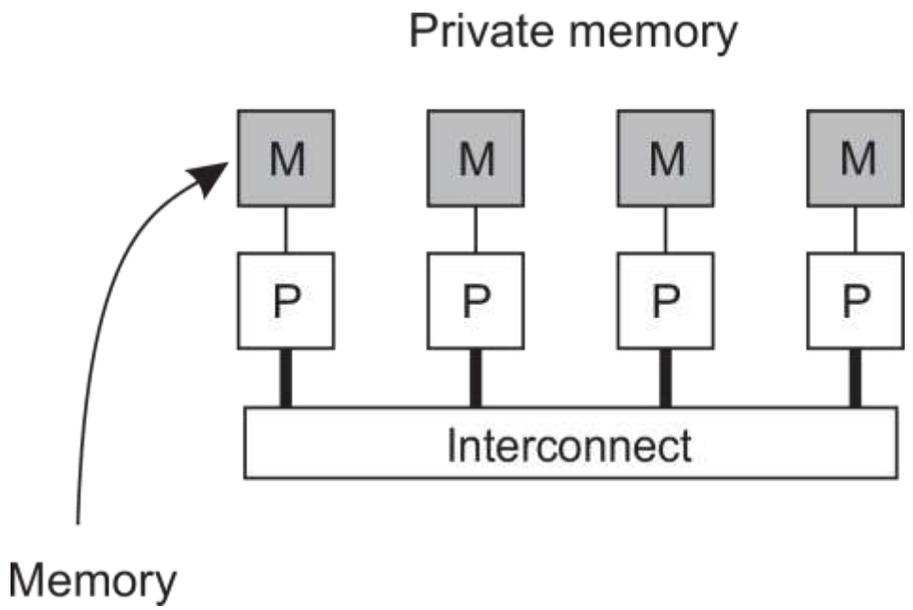
Observation

High-performance distributed computing started with parallel computing

Multiprocessor and multicore versus multicomputer



Multiple CPUs are organized in such a way that they all have access to the same physical memory. The shared-memory model turned out to be highly convenient for improving the performance of programs, and it was relatively easy to program.



Multiple CPUs are organized in such a way that they all have access to the same physical memory.

Distributed Shared Memory Systems

Observation

Multiprocessors are relatively easy to program in comparison to multicomputers, yet have problems when increasing the number of processors (or cores).

Solution: Try to implement a shared-memory model on top of a multicomputer.

Example through virtual-memory techniques

Map all main-memory pages (from different processors) into one single virtual address space. If a process at processor *A* addresses a page *P* located at processor *B*, the OS at *A* traps and fetches *P* from *B*, just as it would if *P* had been located on local disk.

Problem

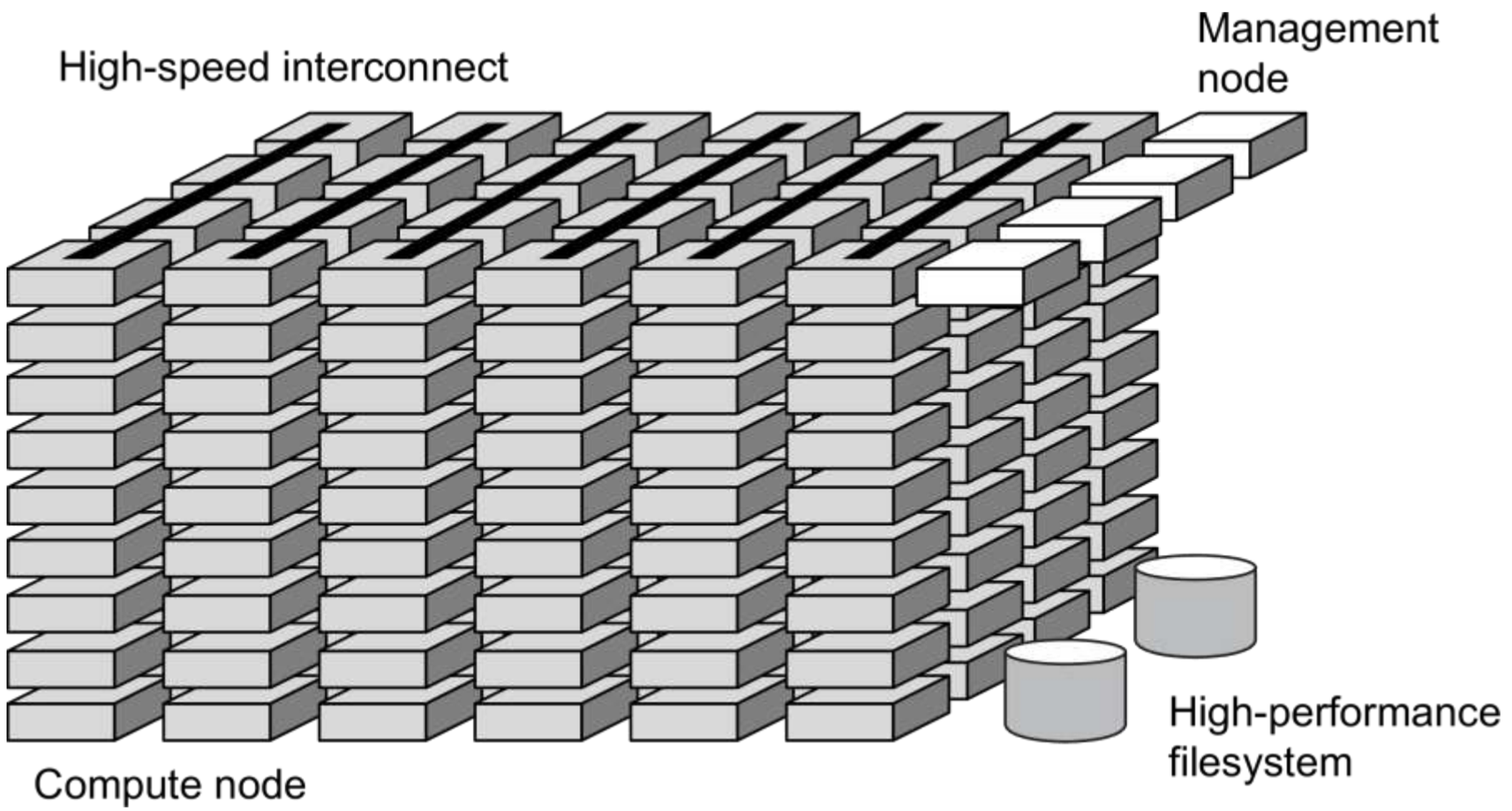
Performance of distributed shared memory could never compete with that of multiprocessors, and failed to meet the expectations of programmers. It has been widely abandoned by now.

High-performance distributed computing

- Cluster Computing

  Essentially a group of high-end systems connected through a LAN

  - Homogeneous: same OS, near-identical hardware
  - Single, or tightly coupled managing node(s)



High-performance distributed computing

- Cluster Computing

It became popular as the price/performance ratio of personal computers and workstations improved significantly. Instead of investing in a traditional supercomputer, it became both financially and technically viable to build a supercomputer by connecting a collection of relatively simple computers via a high-speed network. This approach is commonly used for parallel programming, where a single compute-intensive program runs simultaneously on multiple machines.

Over time, the development of supercomputers organized as clusters has advanced to the point where we now see clusters with over 100,000 CPUs, each CPU having 8 or 16 cores. These clusters utilize multiple networks. The most critical network is a high-speed interconnect network that links the various nodes together. Additionally, there is a separate management network used to monitor and control the system's organization and performance. Furthermore, these systems often use a special high-performance file system or database with its own dedicated local network. The figure, however, does not show additional equipment like high-speed I/O and networking facilities for remote access and communication.

High-performance distributed computing

- Cluster Computing

A management node typically collects jobs from users and then distributes the associated tasks among the various compute nodes. For very large clusters, several management nodes are used to handle the load. These management nodes run the necessary software for executing programs and managing the cluster, while the compute nodes are equipped with a standard operating system that has been extended with additional functions for communication, storage, fault tolerance, and more.

A significant development in this field is the evolution of the operating system. There has been a clear trend towards minimizing the operating system to lightweight kernels to ensure the least possible overhead. However, the drawback of such specialized operating systems is that they become highly fine-tuned to the underlying hardware, which affects compatibility and openness. To address this, multi-kernel approaches are being developed. In a multi-kernel system, a full-fledged operating system operates alongside a lightweight kernel, achieving the benefits of both worlds. This setup is increasingly necessary as high-performance compute nodes often need to run multiple independent jobs simultaneously.

High-performance distributed computing

- Grid Computing

The next step: plenty of nodes from everywhere

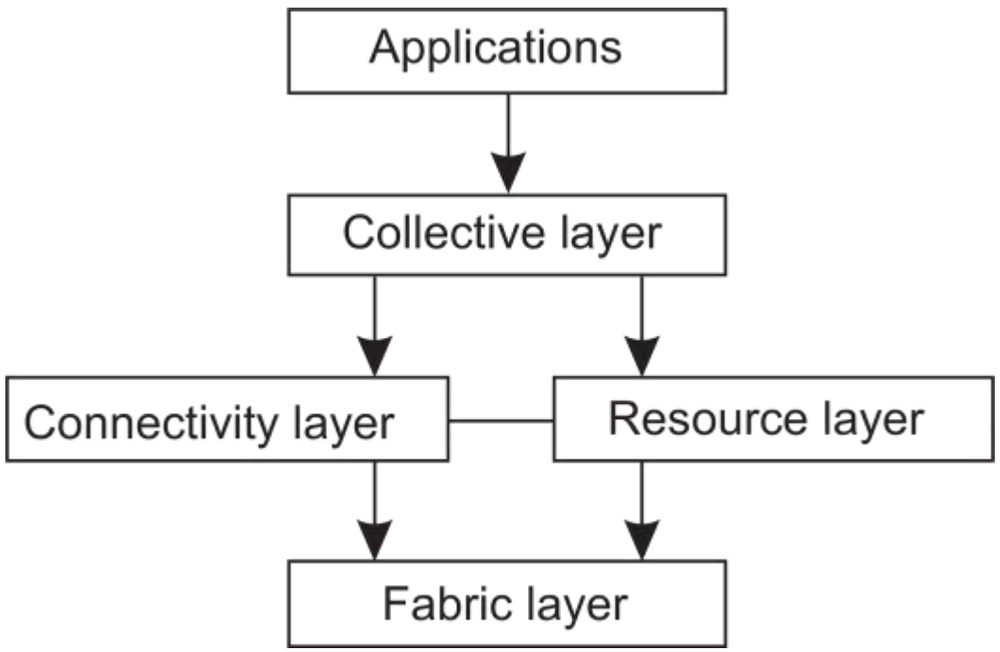
- Heterogeneous
- Dispersed across several organizations
- Can easily span a wide-area network

Note

To allow for collaborations, grids generally use virtual organizations. In essence, this is a grouping of users (or better: their IDs) that allows for authorization on resource allocation.

High-performance distributed computing

- Grid Computing: Architecture



The layers

- Fabric: Provides interfaces to local resources (for querying state and capabilities, locking, etc.)
- Connectivity: Communication/transaction protocols, e.g., for moving data between resources. Also various authentication protocols.
- Resource: Manages a single resource, such as creating processes or reading data.
- Collective: Handles access to multiple resources: discovery, scheduling, replication.
- Application: Contains actual grid applications in a single organization.

Distributed information system

- Integrating Applications

Situation

Organizations confronted with many networked applications, but achieving interoperability was painful.

Basic approach

A networked application is one that runs on a server making its services available to remote clients.

Simple integration: clients combine requests for (different) applications; send that off; collect responses, and present a coherent result to the user.

Next step

Allow direct application-to-application communication, leading to Enterprise Application Integration (EAI).

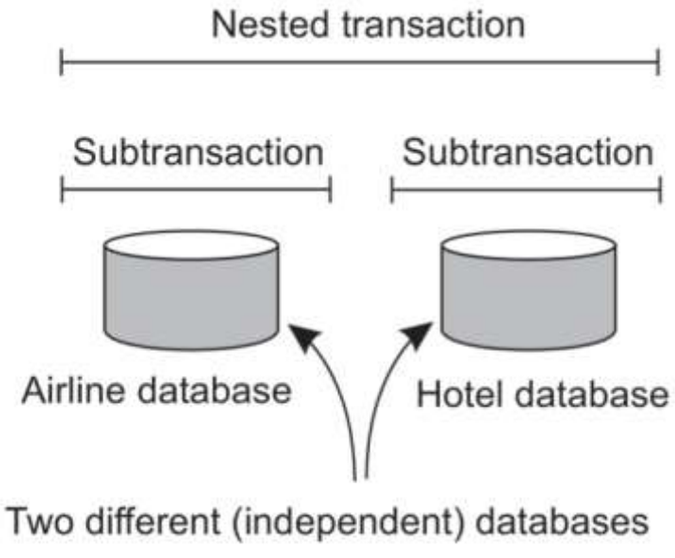
Distributed information system

- Distributed transaction processing

Example EAI: (nested) transactions

Primitive	Description
BEGIN_TRANSACTION	Mark the start of a transaction
END_TRANSACTION	Terminate the transaction and try to commit
ABORT_TRANSACTION	Kill the transaction and restore the old values
READ	Read data from a file, a table, or otherwise
WRITE	Write data to a file, a table, or otherwise

Issue: all-or-nothing: The characteristic feature of a transaction is either all of these operations are executed or none are executed.

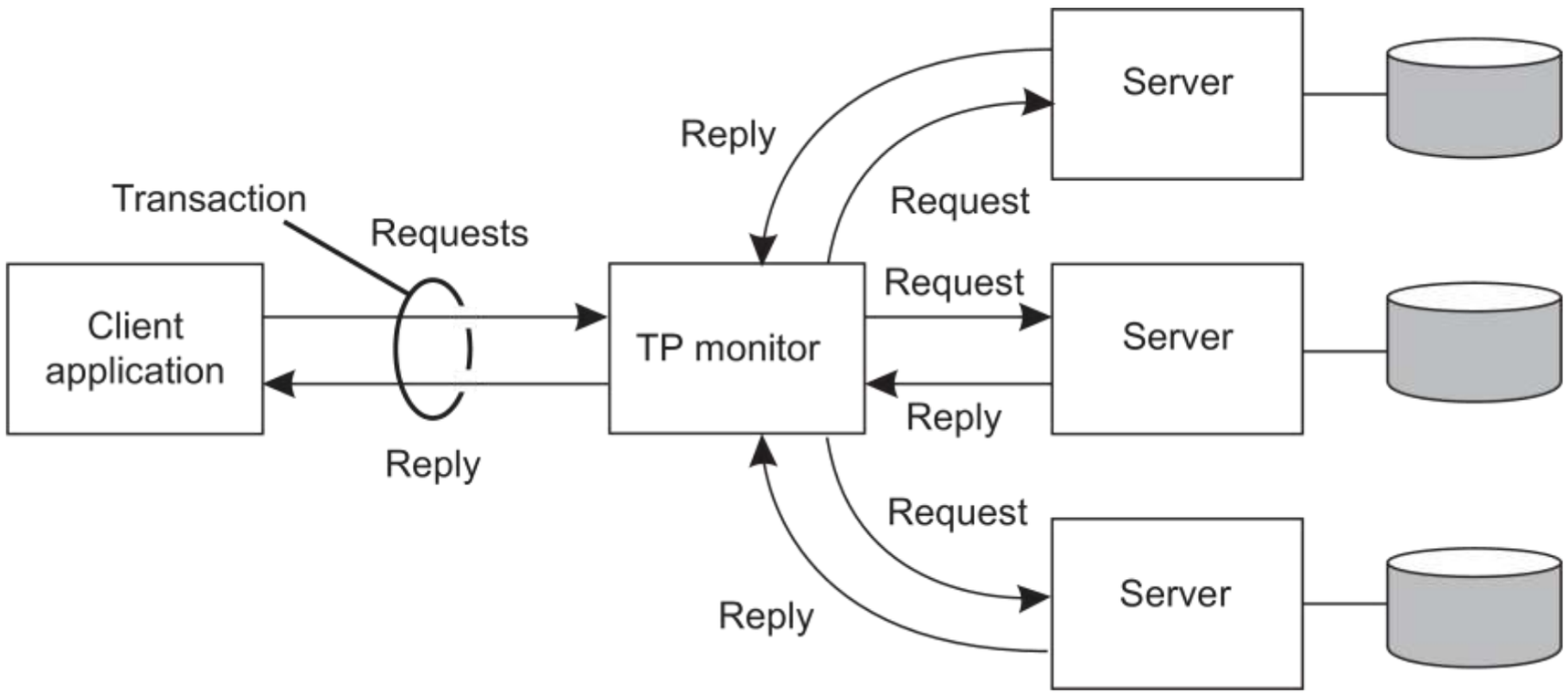


- Atomic: happens indivisibly (seemingly)
- Consistent: does not violate system invariants
- Isolated: not mutual interference
- Durable: commit means changes are permanent

Distributed information system

TPM: Transaction Processing Monitor

The component that handles distributed (or nested) transactions belongs to the core for integrating applications at the server or database level. Its main task is to allow an application to access multiple server/databases by offering it a transactional programming

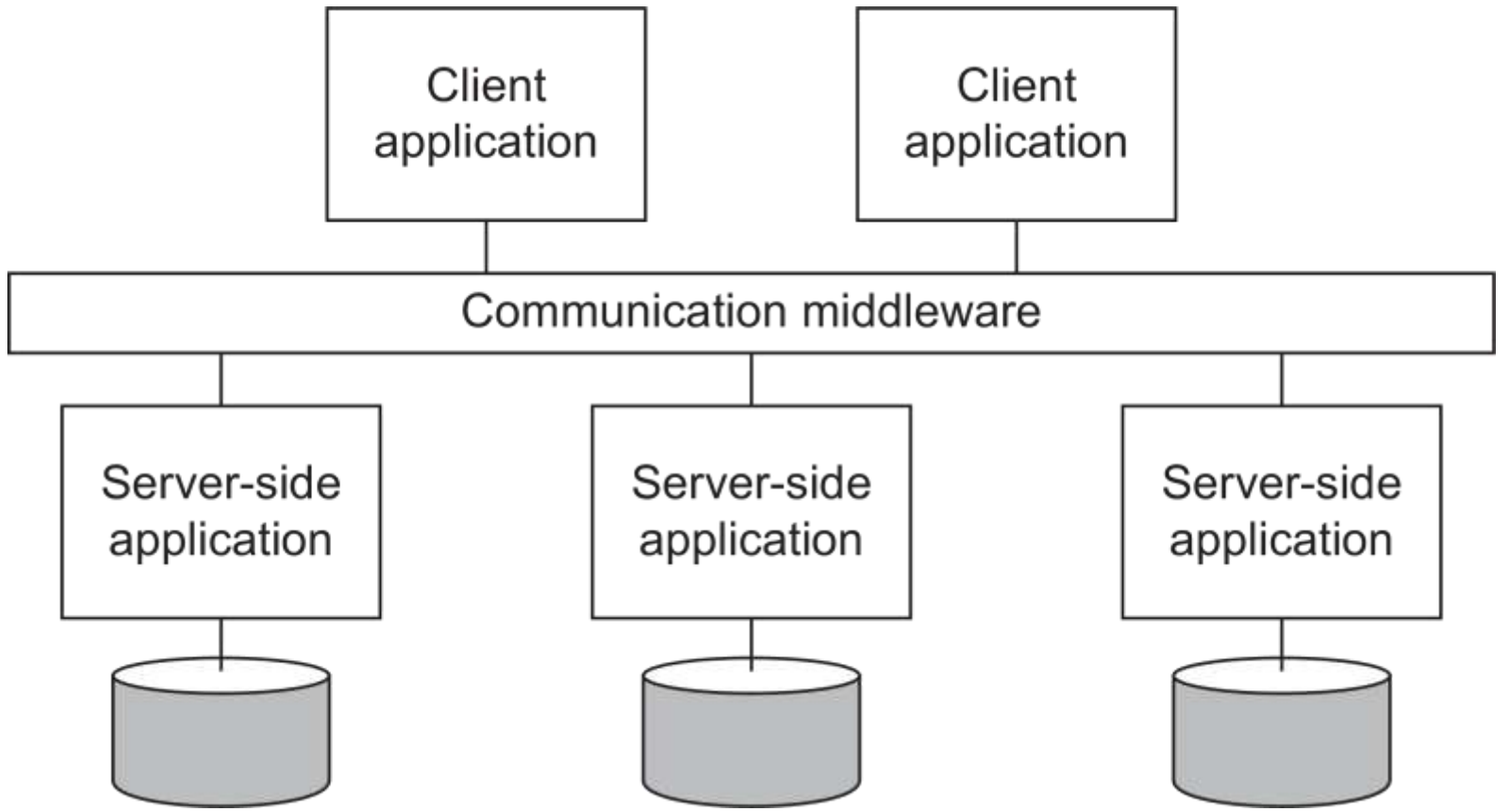


Observation

Often, the data involved in a transaction is distributed across several servers. A TP Monitor is responsible for coordinating the execution of a transaction.

Distributed information system

Middleware and EAI



Middleware offers communication facilities for integration

Remote Procedure Call (RPC): Requests are sent through local procedure call, packaged as message, processed, responded through message, and result returned as return from call.

Message Oriented Middleware (MOM): Messages are sent to logical contact point (published), and forwarded to subscribed applications.

Distributed information system

How to integrate applications

File transfer: Technically simple, but not flexible:

- Figure out file format and layout
- Figure out file management
- Update propagation, and update notifications.

Shared database: Much more flexible, but still requires common data scheme next to risk of bottleneck.

Remote procedure call: Effective when execution of a series of actions is needed.

Messaging: RPCs require caller and callee to be up and running at the same time. Messaging allows decoupling in time and space.

Distributed pervasive systems

Observation

Emerging next-generation of distributed systems in which nodes are small, mobile, and often embedded in a larger system, characterized by the fact that the system naturally blends into the user's environment.

Three (overlapping) subtypes

- Ubiquitous computing systems: pervasive and continuously present, i.e., there is a continuous interaction between system and user.
- Mobile computing systems: pervasive, but emphasis is on the fact that devices are inherently mobile.
- Sensor (and actuator) networks: pervasive, with emphasis on the actual (collaborative) sensing and actuation of the environment.

Distributed pervasive systems

- Ubiquitous computing systems

Core elements

1. (Distribution) Devices are networked, distributed, and accessible transparently
2. (Interaction) Interaction between users and devices is highly unobtrusive
3. (Context awareness) The system is aware of a user's context to optimize interaction
4. (Autonomy) Devices operate autonomously without human intervention, and are thus highly self-managed
5. (Intelligence) The system as a whole can handle a wide range of dynamic actions and interactions

Distributed pervasive systems

- Mobile computing systems

Distinctive features

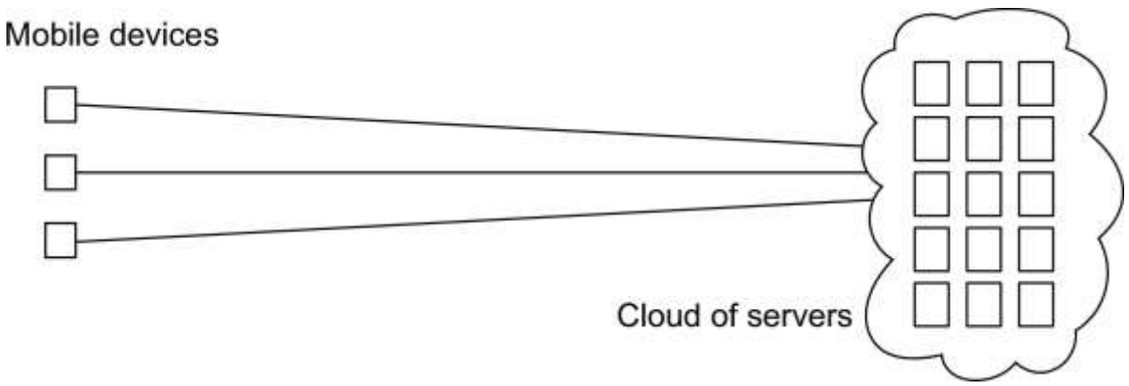
- A myriad of different mobile devices (smartphones, tablets, GPS devices, remote controls, active badges).
- Mobile implies that a device's location is expected to change over time ⇒ change of local services, reachability, etc. Keyword: discovery.
- Maintaining stable communication can introduce serious problems.
- For a long time, research has focused on directly sharing resources between mobile devices. It never became popular and is by now considered to be a fruitless path for research.

Bottomline

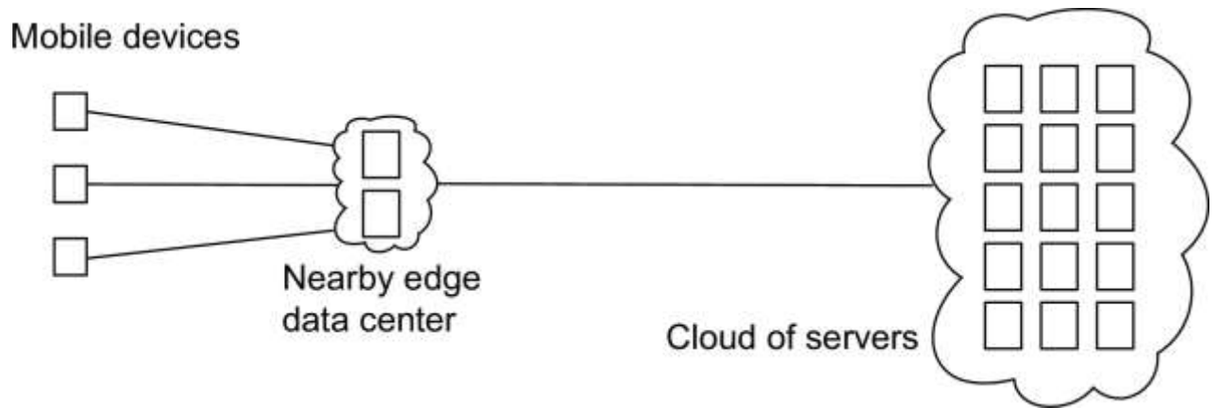
Mobile devices set up connections to stationary servers, essentially bringing mobile computing in the position of clients of cloud-based services.

Distributed pervasive systems

- Mobile computing systems



Mobile cloud computing



Mobile edge computing

There are two mobile computing systems, which are mobile cloud computing and mobile edge computing, or simply MEC, in contrast to Mobile Cloud Computing (MCC). MEC is becoming increasingly important in those cases where latency and computational issues play a role for the mobile device. Typical example applications that require short latencies and computational resources include augmented reality, interactive gaming, real-time sports monitoring, and various health applications. In these examples, the combination of monitoring, analyses, and immediate feedback in general make it difficult to rely on servers that may be placed thousands of miles from the mobile devices.

Distributed pervasive systems

- Sensor networks

  ## Characteristics

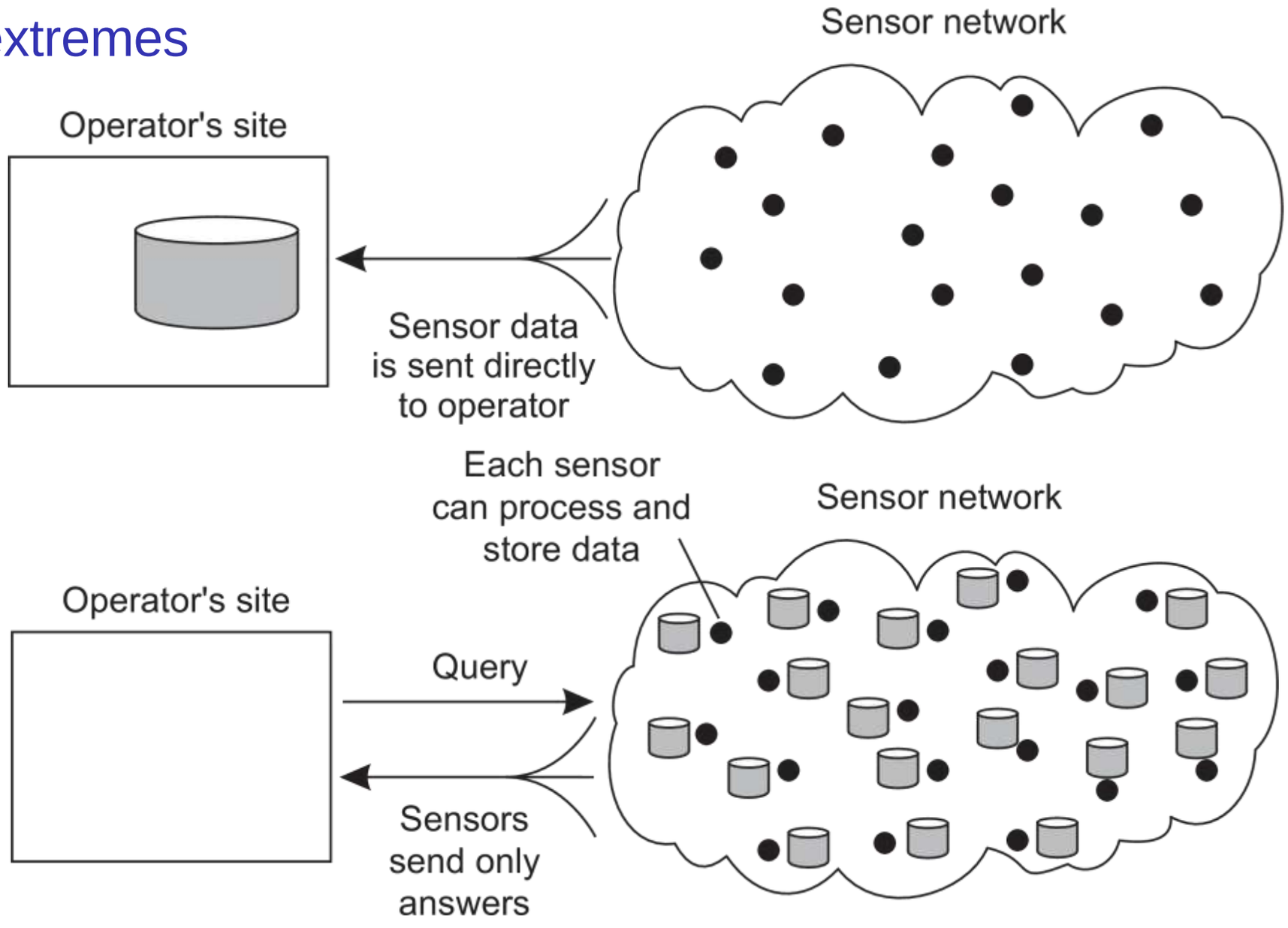
  The nodes to which sensors are attached are:

  - Many (10s-1000s)
  - Simple (small memory/compute/communication capacity)
  - Often battery-powered (or even battery-less)

Distributed pervasive systems

- Sensor networks as distributed database

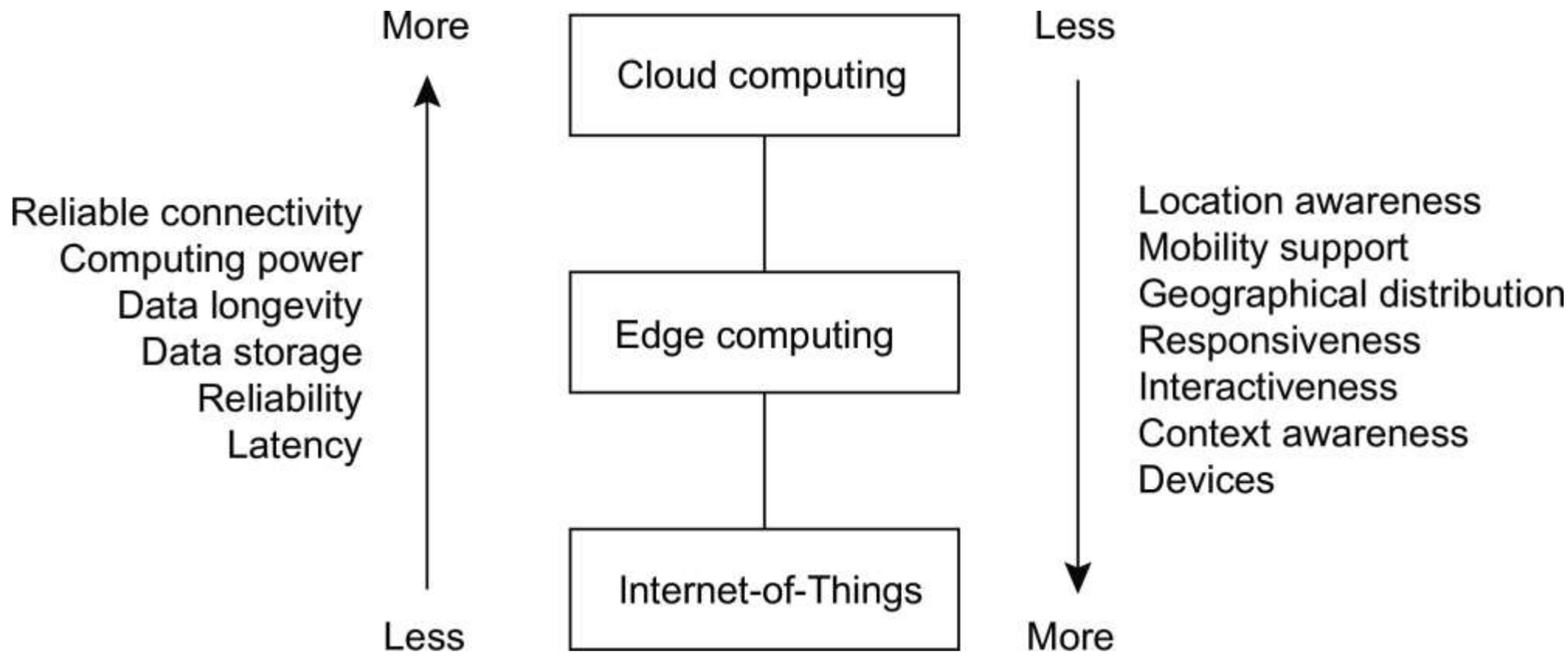
Two extremes



Distributed pervasive systems

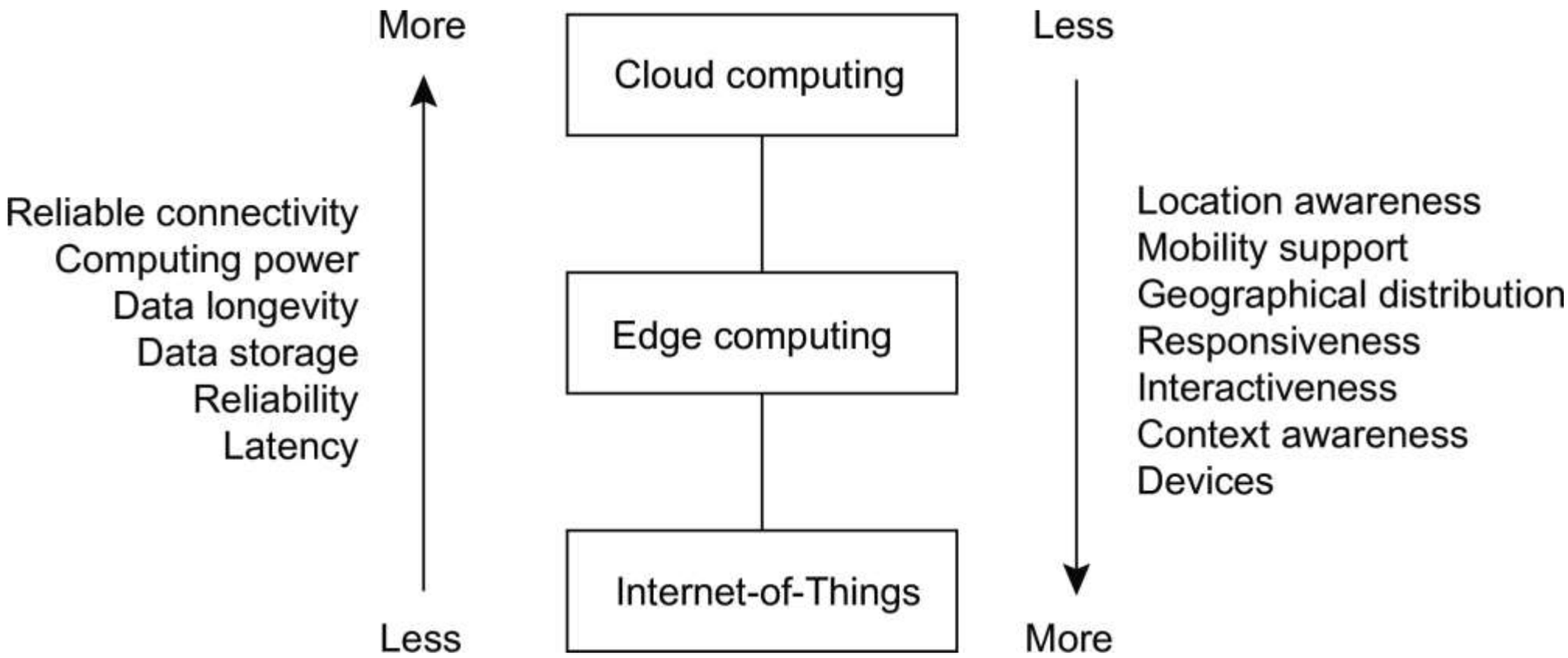
- Cloud, edge, things

Distributed systems cover a wide range of different networked computer systems. Many of these systems operate with computers connected through a local-area network, or LAN. However, with the growth of the Internet of Things (IoT) and the increased connectivity to remote services offered by cloud-based systems, we are seeing new types of organizations across wide-area networks, or WANs, emerging.



Distributed pervasive systems

- Cloud, edge, things



Typically, higher up the hierarchy we see that typical qualities of distributed systems improve: they become more reliable, have more capacity, and, in general, perform better. Lower in the hierarchy, we see that location-related aspects are easier facilitated, as well as performance qualities related to latencies. At the same time, the lower parts show in an increase in the number of devices and computers, whereas higher up, the number of computers becomes less.

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many false assumptions are often made.

False (and often hidden) assumptions

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

These assumptions relate to properties unique to distributed systems: reliability, security, heterogeneity, and the network's topology; latency and bandwidth; transport costs; and administrative domains. When developing non-distributed applications, most of these issues will most likely not show up.

Thank You